Set up development environment with .env file containing the Gemini

API key.

Slide 1: Get Your Gemini API Key

- Visit Google AI Studio and sign in with your Google account. [1, 2]
- Click **Create API Key** in the side menu. [1]
- Copy your unique API key and keep it secret. [1, 2]

Slide 2: Create the `.env` File

- Open your project folder in your code editor. [1]
- Create a new file named exactly `.env` in the root directory. [1]
- Add the following line inside the file: [1]

  env
  ```
  GEMINI_API_KEY=your_actual_api_key_here
  ```
  Use code with caution.
- Add `.env` to your `.gitignore` file to protect your key. [1]

Slide 3: Load Variables in Code

- Install the required python packages in your terminal: [1]

  bash
  ```
  pip install google-generativeai python-dotenv
  ```
  Use code with caution.
- Load and use the key in your code: [1, 2]

  python
  ```
  import os
  import google.generativeai as genai
  from dotenv import load_dotenv

  load_dotenv()  # Loads variables from .env
  genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
  ```
  Use code with caution.

  If you want, tell me:

- What **programming language** are you using?
- Which **Gemini model** do you plan to call?

Here is the continuation to expand your presentation slides, covering **project structure**, **running the code**, and **security best practices**.

Slide 4: Project Directory Structure

- Create a clean project folder to keep your files organized.
- Ensure the `.env` file sits at the very root level next to your main script.
- **Recommended Structure:**

  text
  ```
  my-gemini-project/
  ├── .env
  ├── .gitignore
  ```

```
├── requirements.txt
└── app.py
```
Use code with caution.

Slide 5: Running Your First Test

- Create your main application file named `app.py`.
- Add a simple test prompt to verify the API key connection works:
  python
  ```
  model = genai.GenerativeModel('gemini-1.5-flash')
  response = model.generate_content("Write a 3-word welcoming slogan.")
  print(response.text)
  ```
  Use code with caution.
- Run the script in your terminal using the command: `python app.py`

Slide 6: Security & Deployment Rules

- **Never commit** your `.env` file to public repositories like GitHub.
- Always add `.env` to your `.gitignore` file before staging changes.
- When deploying to cloud platforms (like Render, Vercel, or AWS), do not upload the file; instead, set `GEMINI_API_KEY` directly in the platform's **Environment Variables** settings.
  If you want, tell me:

- Do you need this exported into an actual **PowerPoint (.pptx) file**?
- What **specific topic** or app are you planning to build next?

Slide 7: Error Handling & API Validation

- Wrap your API calls in `try-except` blocks to handle network issues gracefully.
- Catch specific exceptions to find out if your API key is invalid or expired.
- **Code Example:**
  python
  ```
  try:
      response = model.generate_content("Hello!")
      print(response.text)
  except Exception as e:
      print(f"API Error: {e}")
  ```
  Use code with caution.

Slide 8: Managing Quotas & Rate Limits

- Keep track of your usage limits inside the **Google AI Studio Dashboard**.
- The **Free Tier** has specific rate limits (e.g., requests per minute) that you must stay within.
- Implement basic delays or exponential backoff in your loops if you hit rate limit errors (HTTP 429).

Slide 9: Optimization with Streaming Responses

- Use streaming to make your application feel faster and more interactive for users.
- Streaming fetches chunks of text as they are generated instead of waiting for the full answer.
- **Code Example:**

  python

  ```
  response = model.generate_content("Write a long poem", stream=True)
  for chunk in response:
      print(chunk.text, end="")
  ```

  Use code with caution.